

Roteamento Dinâmico com Containers Docker

Comparativo entre RIP, OSPF e Algoritmo Próprio

Objetivo do Trabalho

Problema e Proposta

Comparar protocolos de roteamento sem infraestrutura física

Virtualizar roteadores reais usando containers Docker

Implementar RIP, OSPF e um algoritmo próprio no mesmo ambiente

Analisar as decisões de caminho de cada abordagem

Ferramentas

Docker Desktop

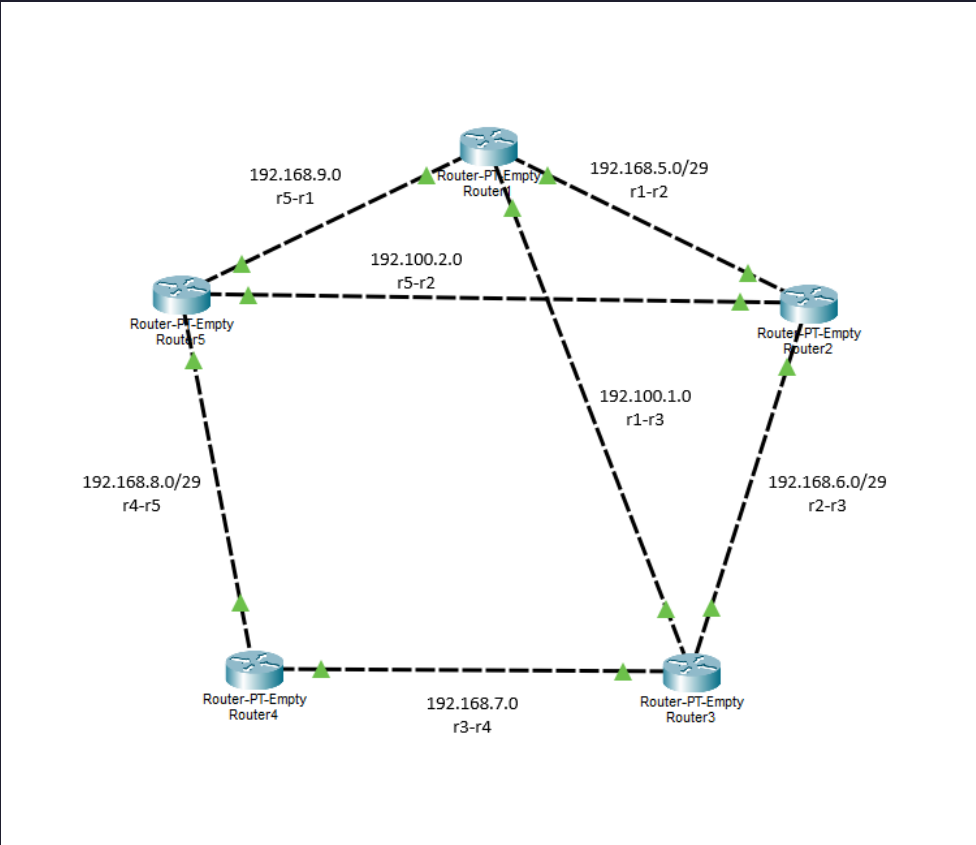
FRRouting (FRR) — daemon de roteamento open-source

Python 3 — algoritmo próprio e geração de configs

Shell script (aplicar.sh) — orquestração

Cada container Docker representa um roteador físico com permissões de rede reais (SYS_ADMIN, NET_ADMIN, ip_forward), executando o mesmo stack de roteamento usado em produção.

Topologia da Rede



Enlace	Rede	Tipo
R1 – R2	192.168.5.0/29	Anel principal
R2 – R3	192.168.6.0/29	Anel principal
R3 – R4	192.168.7.0/29	Anel principal
R4 – R5	192.168.8.0/29	Anel principal
R5 – R1	192.168.9.0/29	Anel principal
R1 – R3	192.100.1.0/29	Redundância
R5 – R2	192.100.2.0/29	Redundância

Infraestrutura

Dockerfile	Imagem Debian + FRR + Python 3 + iproute2 + ping + traceroute
docker-compose.yaml	5 serviços com IPs fixos, permissões NET_ADMIN e ip_forward=1
aplicar.sh	Copia configs e reinicia FRR (rip/ospf) ou inicia daemon Python (proprio)
configs/rip/	frr.conf com router rip para cada roteador
configs/ospf/	frr.conf com router ospf e router-id único por roteador
algoritmo_proprio.py	Agente Link-State: HELLO, LSA, Dijkstra — copiado para /opt/
main.py	Orquestrador de sockets UDP: dispara threads e liga o AgentLinkState
<pre>./aplicar.sh rip # aplica RIP em todos os roteadores ./aplicar.sh ospf # aplica OSPF ./aplicar.sh proprio # para FRR, copia daemon Python e inicia em cada container</pre>	

RIP — Routing Information Protocol

Como funciona

Cada roteador anuncia suas redes aos vizinhos

Métrica = número de saltos (máx. 15)

Atualização periódica a cada 30 segundos

DA = 120 | Daemon: ripd

```
router rip
network 192.168.5.0/29
network 192.168.9.0/29
network 192.100.1.0/29
redistribute connected
version 2
```

Tabela de rotas — R1 (output real)

R>* 192.168.6.0/29 [120/2] via 192.168.5.3 ← foi por R2

R>* 192.168.7.0/29 [120/2] via 192.100.1.3 ← foi por R3

R>* 192.168.8.0/29 [120/2] via 192.168.9.3 ← foi por R5

R4→R1-R2: [120/3] ← 3 saltos (maior distância)

Características

Convergência: 30–90 segundos

Links rápidos ignorados se não reduzem saltos

Simples de configurar e entender

OSPF — Open Shortest Path First

Como funciona

Cada roteador constrói mapa completo da rede (LSA)

Roda Dijkstra localmente para calcular melhores rotas

Convergência rápida (~5-15s) via troca de LSAs

DA = 110 | Daemon: ospfd | Router-ID único por roteador

```
router ospf
ospf router-id 1.1.1.1
network 192.100.1.0/29 area 0
network 192.168.5.0/29 area 0
network 192.168.9.0/29 area 0
```

Tabela de rotas — R5 (output real)

O>* 192.100.1.0/29 [110/20] via 192.168.9.2 ← remota, custo 20

O>* 192.168.5.0/29 [110/20] via 192.168.9.2 ← 2 hops × 10

O 192.100.2.0/29 [110/10] is directly connected ← perde para C

C>* 192.100.2.0/29 is directly connected ← C (DA=0) ganha de O (DA=110)

Características

Com custo padrão = comportamento similar ao RIP

Potencial real: customizar custo por interface

Rotas diretas aparecem duplicadas: O + C>*

Algoritmo Próprio

Custo por enlace = 10 + latência_ms (medida por ping entre vizinhos)

Fluxo do daemon (por container)

1. HELLO (broadcast UDP) — descobre vizinhos diretos
2. Ping — mede latência real a cada vizinho (a cada 10 s)
3. LSA (flooding UDP) — inunda LSDB para toda a rede
4. Dijkstra local — calcula melhor caminho em cada roteador
5. ip route replace — instala rotas no kernel Linux

Características

FRR é parado, então Python assume o kernel
Cada container roda main.py com threads independentes
Reage a mudanças de latência
Timeout de vizinho: 30 s
Timeout de LSA: 60 s (remove info desatualizada)
Portas UDP: 5006 (HELLO) e 5007 (LSA)

```
# kernel route instalada pelo daemon (ip route show):  
192.168.6.0/29 via 192.100.1.3 ← R1 foi por R3 (3 ms, custo 13)  
192.168.5.0/29 via 192.100.2.3 ← R5 foi por R2 (3 ms, custo 13)
```

Comparativo — Decisões de Rota

Origem → Destino	RIP	OSPF	Algoritmo Próprio
R1 → rede R2-R3	via R2 (10ms)	via R2 (custo 10)	★ via R3 (3ms)
R3 → rede R1-R2	via R2	via R2	★ via R1 (3ms)
R5 → rede R1-R2	via R1 (9ms)	via R1 (custo 10)	★ via R2 (3ms)
R4 → rede R1-R2	via R3 [3 saltos]	via R3 [custo 30]	via R3 [custo 25]

Uso dos enlaces de redundância (R1-R3 e R5-R2):

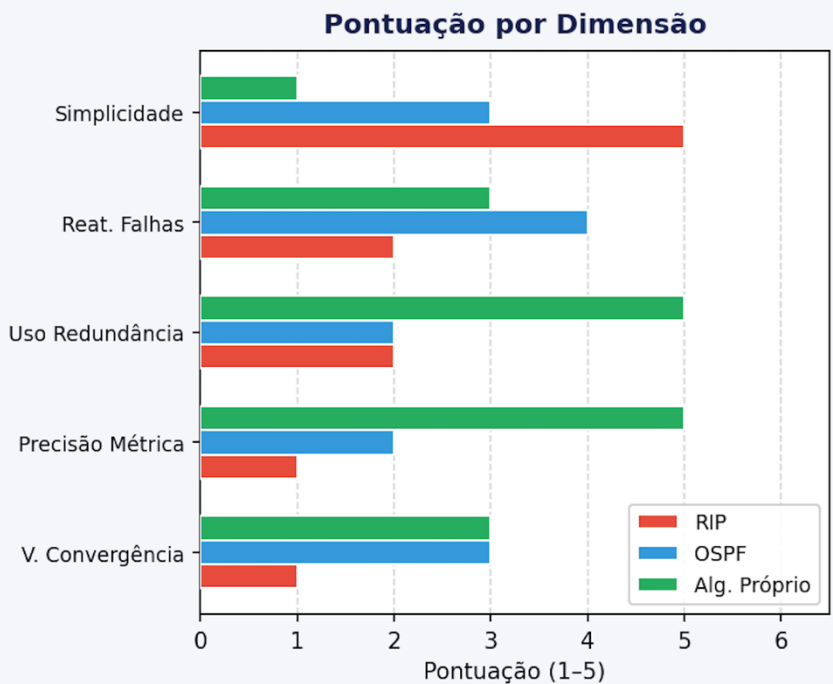
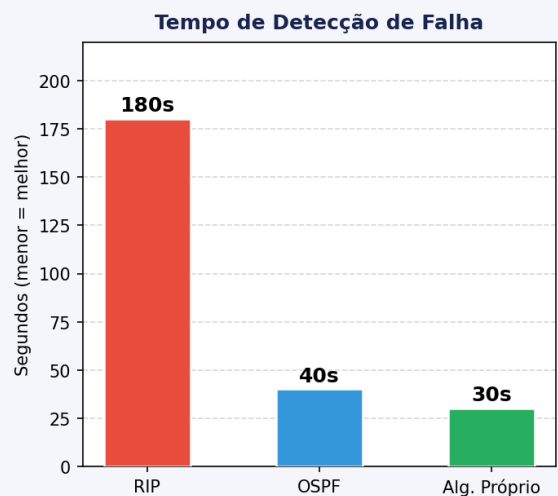
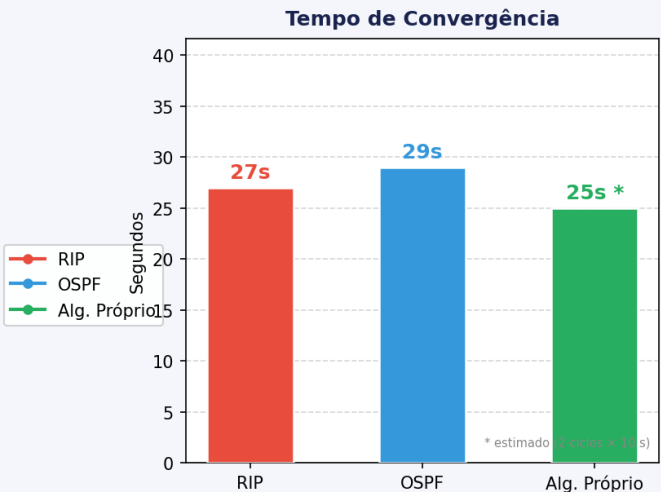
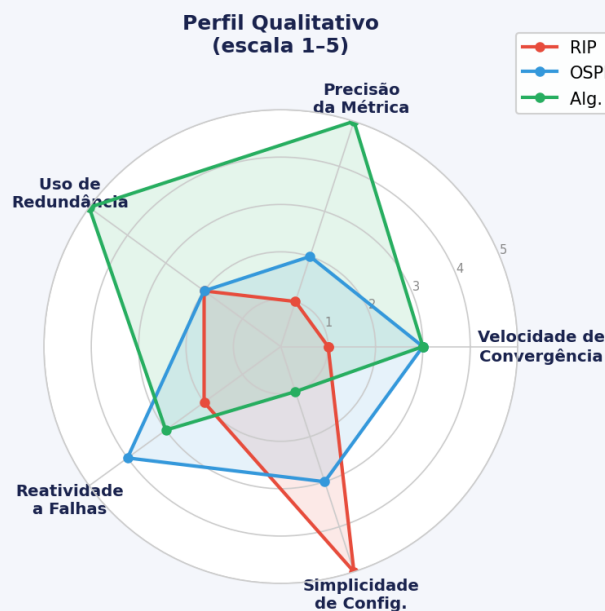
Protocolo	Usa links rápidos?	Critério de decisão
RIP	Parcialmente — por coincidência	Só quando reduz o nº de saltos
OSPF	Parcialmente — custo igual	Custo padrão 10 = igual a todos os enlaces
Próprio	✓ Sistemáticamente	Latência menor = custo menor = rota preferida

Comparativo — Características Técnicas

Critério	RIP	OSPF	Algoritmo Próprio
Código na tabela	R	O	(kernel, ip route)
Métrica	Salto	Custo (padrão 10)	(10 + latência real)
Convergência	30–90 s	~5–30 s	~20–30 s (HELLO-LSA-Dijkstra)
Reage a falhas?	Sim (lento)	Sim (rápido)	Sim — timeout vizinho 30 s
Troca mensagens?	Sim	Sim	Sim (HELLO UDP + LSA flooding)

Comparativo — Gráficos

Comparativo de Características Técnicas — Trabalho_2026



Conclusão

RIP

- Simples, mas é cego à qualidade dos enlaces.
- Convergência lenta. Adequado para redes pequenas e homogêneas.

OSPF

- Mais robusto e de convergência rápida.
- Com custos padrão se comporta como RIP — seu potencial real exige customização de custo por interface.

Algoritmo Próprio

- Daemon Python vivo com HELLO (UDP em Broadcast) + LSA flooding + Dijkstra por container.
- Explora sistematicamente os enlaces rápidos via latência medida.
- Reage a falhas (timeout 30 s). Convergência similar ao OSPF (~20-30 s).